LAB 2:

Surprise CF

MovieLens-100K + Surprise

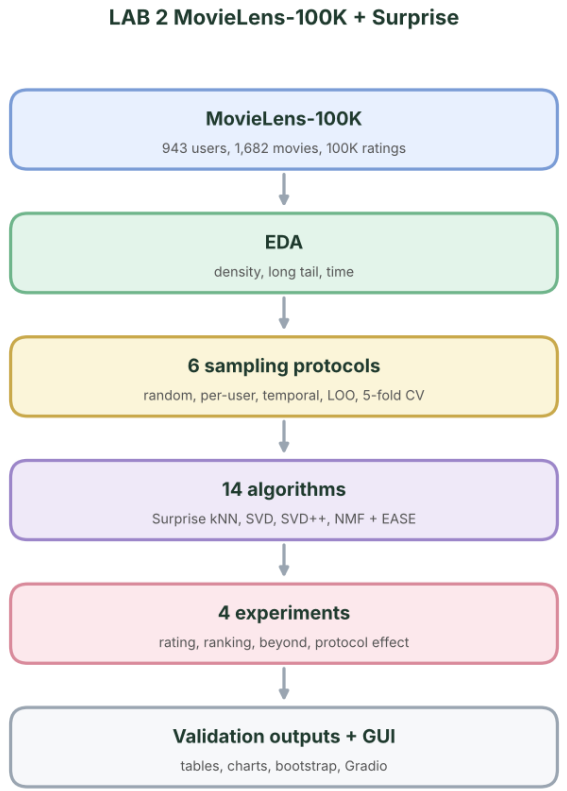
204729

Recommender Systems

ผู้ช่วยศาสตราจารย์ ดร.ดุษฎี ประเสริฐธิติพงษ์

MovieLens-100K - 14 Algorithms

Offline Protocols - GUI



อธิบายแนวคิดของ Source Code ทีละ Part

Sampling - Surprise - Metrics - Bootstrap - Gradio

COMPUTER SCIENCE

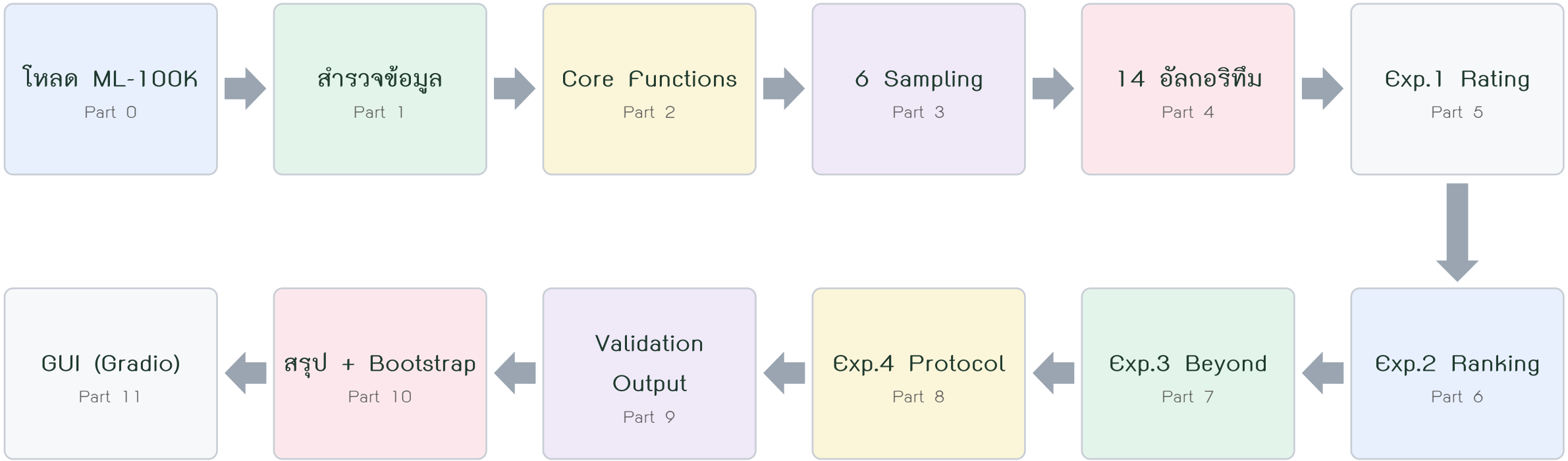
LAB 2 คืออะไร

ภาพรวม

- ขยาย LAB 1 จากข้อมูลของเล่นไปสู่ **Open Dataset จริง: MovieLens-100K** (943 ผู้ใช้ • 1,682 เรื่อง • 100,000 คะแนน)
- ใช้ Library **Surprise** ซึ่งรวมอัลกอริทึมการกรองร่วมของบทที่ 5 ไว้ครบ: kNN หลายแบบ, BaselineOnly, SVD, SVD++, NMF, SlopeOne, CoClustering
- เพิ่ม **Popularity** และ **EASE** ที่เขียนเอง เพื่อเทียบฝั่ง ranking ของข้อมูลโดยปริยาย
- เปรียบเทียบ **6 วิธีเลือกกลุ่มตัวอย่าง** สำหรับ Offline Training/Testing: Random, Per-user, Global Temporal, Per-user Temporal, Leave-One-Liked-Out, 5-Fold CV
- 4 การทดลอง: Rating • Ranking (Full vs Sampled) • Beyond-Accuracy • ผลของ Protocol + Paired Bootstrap
- ตัวอย่าง Output บน Validation เทียบทุกอัลกอริทึม และ **GUI** แสดงข้อมูลผู้ใช้ (อายุ เพศ อาชีพ) พร้อมรายการแนะนำ

ไฟล์: 02_Lab_CF_MovieLens_Surprise_Comparison_Offline_GUI.ipynb • รันบน Colab ราว 8–12 นาที

โครงสร้าง Notebook LAB 2 (Part 0 → Part 11)



Core Functions ห่อ Surprise ให้มี interface เดียวกับ LAB 1: fit • predict • scores

แผนที่ของ Source Code LAB 2

Part	หน้าที่	ฟังก์ชัน / ตัวแปรสำคัญ	ผลลัพธ์
0	โหลดข้อมูล 3 ตาราง	`load_movielens_100k`, `TITLE`, `GENRE`	ratings / items / users
1	EDA	`eda`, `per_user`, `per_item`	density 0.063, กราฟ 4 รูป
2	Core Functions	`SurpriseModel`, `EASEModel`, `evaluate_ranking`	เครื่องมือประเมินผล
3	Sampling	`split_*`, `SPLITS`, `split_table`	5 split + 5-fold CV
4	อัลกอริทึม	`make_models`, `RATING_MODELS`	14 โมเดล
5–8	การทดลอง 1–4	`rating_results`, `cv_results`, `ranking_results`, `protocol_results`	ตารางผล
9	Output บน Validation	`SAMPLE_USERS`, `RANK_LISTS`	ตารางเทียบรายผู้ใช้
10	สรุป + สถิติ	`summary`, `paired_bootstrap`	chart 6 รูป, CI
11	GUI	`gui_main`, `gui_predict`, `gr.Blocks`	แอป 2 แท็บ

Part 0: ติดตั้ง Surprise และ import

```
!pip -q install scikit-surprise gradio

import math, random, warnings, itertools, time, re
from pathlib import Path
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from IPython.display import display
from sklearn.metrics import roc_auc_score

from surprise import (Dataset, Reader, accuracy,
    NormalPredictor, BaselineOnly, KNNBasic, KNNWithMeans,
                     KNNWithZScore, KNNBaseline, SVD,
                         SVDpp, NMF, SlopeOne,
                         CoClustering)
from surprise.model_selection import train_test_split,
    cross_validate

warnings.filterwarnings("ignore")
pd.set_option("display.width", 220);
    pd.set_option("display.max_columns", 40)
SEED = 42
np.random.seed(SEED); random.seed(SEED)
```

อธิบายโค้ด

เตรียม environment

- `scikit-surprise` คอมไพล์ครั้งแรกบน Colab ราว 1–2 นาที
- import อัลกอริทึม 11 คลาสของ Surprise และ `train_test_split`, `cross_validate`
- `roc_auc_score` จาก sklearn ใช้คำนวณ AUC ของ ranking
- `SEED = 42` ใช้ทั้ง numpy, random และทุกอัลกอริทึม

  ⟶ ทำซ้ำได้

Part 0: โหลด MovieLens-100K (1/2)

```
GENRES = ["unknown", "Action", "Adventure", "Animation",
    "Children's", "Comedy", "Crime", "Documentary", "Drama",
          "Fantasy", "Film-Noir", "Horror", "Musical", "Mystery",
             "Romance", "Sci-Fi", "Thriller", "War", "Western"]

def load_movielens_100k():
    """โหลด MovieLens-100K ผ่าน Surprise (ดาวน์โหลดจาก GroupLens
ครั้งแรก)
แล้วอ่าน u.item (ชื่อ/ปี/แนว) และ u.user (อายุ/เพศ/อาชีพ)
จากโฟลเดอร์เดียวกัน
ถ้าดาวน์โหลดไม่ได้จะใช้สำเนาจาก GitHub (RecBole) เป็นทางสำรอง"""
    base = Path.home() / ".surprise_data" / "ml-100k" / "ml-100k"
    try:
        Dataset.load_builtin("ml-100k", prompt=False)
    except Exception as e:
        print("load_builtin ไม่สำเร็จ:", type(e).__name__)
    if (base / "u.data").exists():
        ratings = pd.read_csv(base / "u.data", sep="\t",
            names=["user_id", "item_id", "rating", "timestamp"])
        it = pd.read_csv(base / "u.item", sep="|", header=None,
            encoding="latin-1")
        items = pd.DataFrame({"item_id": it[0], "title": it[1]})
        flags = it.iloc[:, 5:24].values
        items["genre"] = ["|".join(g for g, f in zip(GENRES, row) if
            f == 1) or "unknown" for row in flags]
        users = pd.read_csv(base / "u.user", sep="|",
            names=["user_id", "age", "gender", "occupation", "zip"])
        src = str(base)
```

อธิบายโค้ด

ใช้ Surprise ดาวน์โหลดครั้งแรก

- `Dataset.load_builtin("ml-100k")` ดาวน์โหลดไปที่ `~/.surprise_data`
- อ่านไฟล์ดิบในโฟลเดอร์เดียวกัน: `u.data` (คะแนน+เวลา), `u.item` (ชื่อ+แนว 19 flag), `u.user` (อายุ เพศ อาชีพ)
- แปลง flag 0/1 ของแนวเป็นข้อความ `Action|Sci-Fi`
- ได้ 3 ตารางที่ใช้ทั้ง notebook

Part 0: โหลด MovieLens-100K (2/2)

```
    else:
        url = "https://raw.githubusercontent.com/RUCAIBox/RecBole/master/dat
            aset/ml-100k/"
        ratings = pd.read_csv(url + "ml-100k.inter", sep="\t");
            ratings.columns = ["user_id", "item_id", "rating", "timestamp"]
        it = pd.read_csv(url + "ml-100k.item", sep="\t", dtype=str);
            it.columns = ["item_id", "title", "year", "genre"]
        it["title"] = it["title"] + " (" + it["year"].fillna("") + ")"
        items = it[["item_id",
            "title"]].assign(genre=it["genre"].fillna("unknown").str.replace
            (" ", "|"))
        users = pd.read_csv(url + "ml-100k.user", sep="\t"); users.columns
            = ["user_id", "age", "gender", "occupation", "zip"]
        src = url
    for df in (ratings, items, users):
        for c in ("user_id", "item_id"):
            if c in df.columns: df[c] = df[c].astype(str)
    ratings["rating"] = ratings["rating"].astype(float)
    items["year"] = items["title"].str.extract(r"\((\d{4})\)\s*$")[0]
    return ratings, items, users, src

ratings, items, users, DATA_SRC = load_movielens_100k()
TITLE = dict(zip(items.item_id, items.title)); GENRE =
    dict(zip(items.item_id, items.genre))
YEAR = dict(zip(items.item_id, items.year))
READER = Reader(rating_scale=(1, 5))
LIKE_T = 4.0                                    # rating >= 4 = "ชอบ"
    (relevant) เหมือน LAB 1

print("Source:", DATA_SRC)
print(f"Users={ratings.user_id.nunique()}
    Items={ratings.item_id.nunique()}  Ratings={len(ratings)}")
```

อธิบายโค้ด

ทางสำรอง + ตัวแปรกลาง

- ถ้าดาวน์โหลดจาก GroupLens ไม่ได้ → อ่านสำเนา ml-100k จาก GitHub (RecBole)
- แปลงรหัสเป็น `str` ให้ตรงกับ Surprise ที่ใช้ raw id เป็นข้อความ
- `TITLE`, `GENRE`, `YEAR` ใช้แสดงผลใน Part 9 และ GUI
- `LIKE_T = 4.0`: นิยาม relevant เหมือน LAB 1

Part 1: สำรวจข้อมูล (EDA)

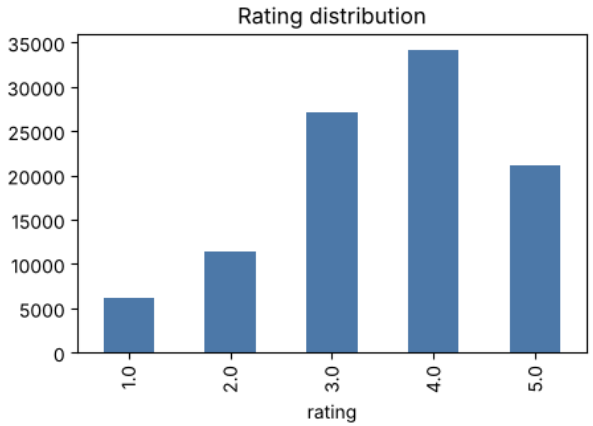
```
n_u, n_i = ratings.user_id.nunique(),
    ratings.item_id.nunique()
per_user = ratings.groupby("user_id").size();
    per_item = ratings.groupby("item_id").size()
ts = pd.to_datetime(ratings.timestamp, unit="s")
eda = pd.Series({
    "density": len(ratings) / (n_u * n_i),
    "global mean": ratings.rating.mean(),
    "ratings/user (median)": per_user.median(),
        "ratings/user (min)": per_user.min(),
    "ratings/item (median)": per_item.median(),
        "items with < 5 ratings": int((per_item <
        5).sum()),
    "share of liked (>=4)": (ratings.rating >=
        LIKE_T).mean(),
    "top 10% items share of ratings":
        per_item.sort_values(ascending=False).head(int
        (0.1 * n_i)).sum() / len(ratings),
    "first date": ts.min().date(), "last date":
        ts.max().date()})
display(eda.to_frame("value"))
```

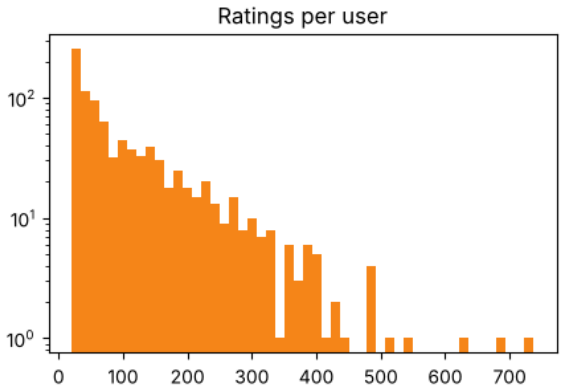
อธิบายโค้ด

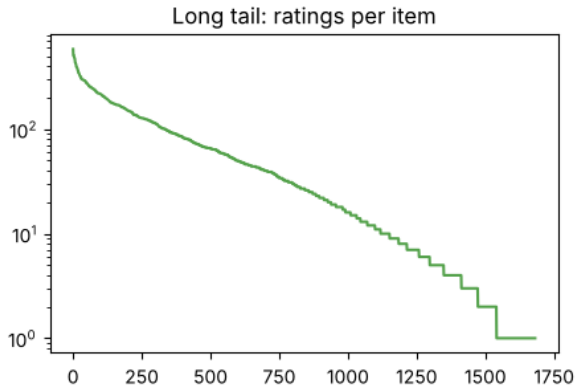
ตัวเลขที่ต้องรู้ก่อนเลือกโมเดล

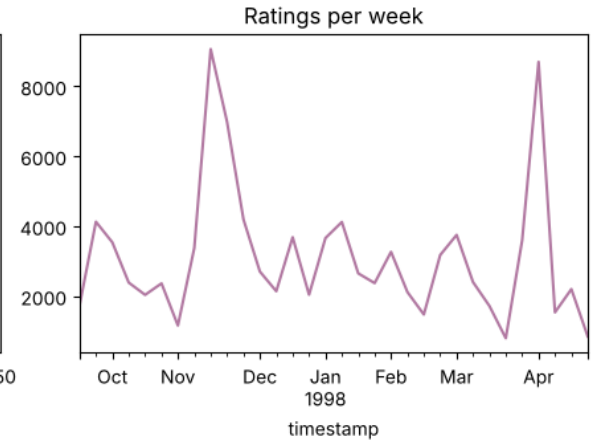
- ความหนาแน่น 0.063 (MovieMini-12 = 0.333)
- ผู้ใช้ทุกคนมี ≥ 20 คะแนน (median 65) แต่ 333 เรื่องมี < 5 คะแนน
- รายการยอดนิยม 10% แรกครองคะแนน 42.7% ⟶ หางยาว
- ชอบ (≥ 4) 55.4% • ข้อมูล 20 ก.ย. 1997 – 22 เม.ย. 1998

Part 1: กราฟสำรวจข้อมูล









- การกระจายคะแนนเบ้ไปทาง 3–4 • จำนวนคะแนนต่อผู้ใช้เบ้ขวา (log scale)
- หางยาว: ไม่กี่เรื่องได้คะแนนหลายร้อย ส่วนใหญ่ได้ไม่กี่คะแนน • กิจกรรมรายสัปดาห์ไม่สม่ำเสมอ ⟶ ต้องระวังการแบ่งตามเวลา

Part 2: แปลง DataFrame และ SurpriseModel

```python
def df_to_trainset(df):
    return Dataset.load_from_df(df[["user_id", "item_id",
        "rating"]], READER).build_full_trainset()

def df_to_testset(df):
    return list(df[["user_id", "item_id",
        "rating"]].itertuples(index=False, name=None))

class SurpriseModel:
    """ห่ออัลกอริทึมของ Surprise ให้มี scores(inner_u)
สำหรับจัดอันดับทุกรายการ"""
    rating_model = True
    def __init__(self, name, factory, family):
        self.name, self.factory, self.family = name, factory,
            family
    def fit(self, trainset):
        self.ts = trainset; self.algo = self.factory();
            self.algo.fit(trainset); return self
    def predict(self, uid, iid):
        return float(self.algo.predict(str(uid), str(iid)).est)
    def scores(self, u):
        a, ts = self.algo, self.ts; mu = ts.global_mean
        # vectorized: mu+bu+bi+qi·(pu+|N(u)|^-1/2 Σ yj)
        if isinstance(a, SVDpp):
            Iu = [j for j, _ in ts.ur[u]]
            imp = a.yj[Iu].sum(0) / np.sqrt(len(Iu)) if Iu else 0
            return mu + a.bu[u] + a.bi + a.qi @ (a.pu[u] + imp)
```

อธิบายโค้ด

ห่อ Surprise ให้จัดอันดับได้เร็ว

- `df_to_trainset` / `df_to_testset`: ใช้ split ที่เขียนเองกับ Surprise ได้
- `predict(uid, iid)` ใช้ raw id ผ่าน `algo.predict`
- `scores(u)` คำนวณทุกรายการแบบ vectorized สำหรับ SVD, SVD++, NMF, BaselineOnly
- SVD++: $\mu + b_u + b_i + q_i \cdot (p_u + |N(u)|^{-1/2} \sum y_j)$

Part 2: กรณีทั่วไป + Popularity

```
        # generic: เรียก estimate ทีละรายการ
        out = np.empty(ts.n_items)
        for i in range(ts.n_items):
            try:
                est = a.estimate(u, i)
                out[i] = est[0] if isinstance(est, tuple)
                    else est
            # PredictionImpossible → ค่าเฉลี่ยรวม
            except Exception:
                out[i] = mu
        return out

class PopularityModel:
    """จำนวนผู้ชอบ (rating>=4) ในข้อมูลฝึก – ranking
อย่างเดียว"""
    rating_model, family = False, "Baseline"
    def __init__(self, name="Popularity"): self.name = name
    def fit(self, ts):
        self.ts = ts; self.pop = np.zeros(ts.n_items)
        for _, i, r in ts.all_ratings():
            if r >= LIKE_T: self.pop[i] += 1
        return self
    def predict(self, uid, iid): return np.nan
    def scores(self, u): return self.pop.copy()
```

อธิบายโค้ด

โมเดลที่ไม่มีสูตร vectorized

- kNN / SlopeOne / CoClustering: เรียก `estimate()` ทีละรายการ
- kNN คืน tuple (est, details) ⟶ ดึงค่าแรก
- ทำนายไม่ได้ (`PredictionImpossible`) ⟶ ใช้ค่าเฉลี่ยรวม เหมือน default ของ Surprise
- PopularityModel: นับผู้ชอบ (≥ 4) ใน trainset ด้วย inner id

Part 2: EASEModel (5.12.2)

```python
class EASEModel:
    """EASE (Steck 2019) บนเมทริกซ์ชอบ 0/1: B =
        -P/diag(P), diag(B)=0"""
    rating_model, family = False, "Implicit"
    def __init__(self, lam=500.0, name="EASE"):
        self.lam, self.name = lam, name
    def fit(self, ts):
        self.ts = ts; X = np.zeros((ts.n_users,
            ts.n_items), dtype=np.float32)
        for u, i, r in ts.all_ratings():
            if r >= LIKE_T: X[u, i] = 1
        P = np.linalg.inv(X.T @ X + self.lam *
            np.eye(ts.n_items))
        B = -P / np.diag(P); np.fill_diagonal(B, 0);
            self.X, self.B = X, B
        return self
    def predict(self, uid, iid): return np.nan
    def scores(self, u): return self.X[u] @ self.B
```

อธิบายโค้ด

ฐานที่แข็งแรงสำหรับ ranking

- สร้างเมทริกซ์ชอบ X (943×1,682) จาก `ts.all_ratings()`
- $P = (X^T X + \lambda I)^{-1} \longrightarrow B = -P/\mathrm{diag}(P),\ \mathrm{diag}(B) = 0$
- $\lambda = 500$ ตามช่วงที่บทแนะนำสำหรับข้อมูลขนาดนี้ (100–1,000)
- `scores(u)` = แถว u ของ X คูณ B — ทั้งหมดเป็นคำตอบรูปปิด ไม่มี SGD

Part 2: Metric ของ Ranking (1/2)

```python
def precision_at_k(recs, rel, k): return
    len(set(recs[:k]) & set(rel)) / k if k else 0.0
def recall_at_k(recs, rel, k): rel = set(rel); return
    len(set(recs[:k]) & rel) / len(rel) if rel else 0.0
def f1_at_k(recs, rel, k):
    p, r = precision_at_k(recs, rel, k),
        recall_at_k(recs, rel, k); return 0.0 if p + r
        == 0 else 2*p*r/(p+r)
def hr_at_k(recs, rel, k): return
    float(bool(set(recs[:k]) & set(rel)))
def reciprocal_rank(recs, rel):
    rel = set(rel)
    for rank, iid in enumerate(recs, 1):
        if iid in rel: return 1.0 / rank
    return 0.0
```

อธิบายโค้ด

ชุดเดียวกับ LAB 1

- Precision@K = ถูก/K • Recall@K = ถูก/จำนวน relevant
- F1@K ค่าเฉลี่ยฮาร์มอนิก • HR@K = มีถูกอย่างน้อย 1 เรื่องหรือไม่
- Reciprocal Rank = 1/อันดับของเรื่องถูกเรื่องแรก (ใช้รายการเต็มทั้งหมด)
- บน MovieLens ผู้ใช้มี relevant หลายเรื่อง จึงใช้สูตรทั่วไป

Part 2: Metric ของ Ranking (2/2) + Rating

```python
def ap_at_k(recs, rel, k):
    rel = set(rel); hits = 0; s = 0.0
    for n, iid in enumerate(recs[:k], 1):
        if iid in rel: hits += 1; s += hits / n
    return s / min(len(rel), k) if rel else 0.0
def ndcg_at_k(recs, rel, k):
    rel = set(rel)
    dcg = sum(1/math.log2(n+2) for n, iid in
        enumerate(recs[:k]) if iid in rel)
    idcg = sum(1/math.log2(n+2) for n in
        range(min(len(rel), k)))
    return dcg / idcg if idcg else 0.0

def rating_metrics(preds):
    """preds = ผลจาก algo.test(): RMSE, MAE, FCP
และสัดส่วนที่ทำนายไม่ได้ (cold start)"""
    impossible = np.mean([p.details.get("was_impossible",
        False) for p in preds])
    return {"RMSE": accuracy.rmse(preds, verbose=False),
        "MAE": accuracy.mae(preds, verbose=False),
            "FCP": accuracy.fcp(preds, verbose=False),
                "Impossible%": 100 * impossible}
```

อธิบายโค้ด

MAP, nDCG และ metric ของ Surprise

- AP@K เฉลี่ย precision ณ ตำแหน่งที่ถูก • nDCG@K ส่วนลด 1/log2
- `rating_metrics` ใช้ `surprise.accuracy`: RMSE, MAE และ FCP (สัดส่วนคู่ที่เรียงถูก)
- Impossible% = คู่ที่ผู้ใช้/รายการไม่อยู่ในชุดฝึก ⟶ วัดปัญหา Cold Start

Part 2: Beyond-Accuracy (1/2)

```
G_INDEX = {g: k for k, g in enumerate(GENRES)}
def genre_vec(iid):
    v = np.zeros(len(GENRES))
    for g in GENRE.get(iid, "unknown").split("|"):
        v[G_INDEX.get(g, 0)] = 1
    return v
GVEC = {iid: genre_vec(iid) for iid in items.item_id}

def coverage(all_recs, n_catalog): return
    len(set(all_recs)) / n_catalog
def gini_coefficient(all_recs, catalog):
    counts = pd.Series(all_recs).value_counts()
    x = np.sort(np.array([counts.get(i, 0) for i in
        catalog], dtype=float)); n = len(x)
    return float((2*np.sum(np.arange(1,
        n+1)*x)/(n*x.sum())) - (n+1)/n) if x.sum() else
        0.0
```

อธิบายโค้ด

ระดับระบบ

- เวกเตอร์แนว 19 มิติต่อเรื่อง (GVEC) สร้างครั้งเดียว ใช้กับ ILD และ Calibration
- Coverage: สัดส่วนเรื่องในคลังของ trainset ที่ถูกแนะนำอย่างน้อยหนึ่งครั้ง
- Gini: นับความถี่ที่แต่ละเรื่องถูกแนะนำ ⟶ 1 = กระจุกที่ไม่กี่เรื่อง

Part 2: Beyond-Accuracy (2/2)

```python
def ild(rec_ids):
    if len(rec_ids) < 2: return 0.0
    V = np.array([GVEC[i] for i in rec_ids]); V = V /
        np.maximum(np.linalg.norm(V, axis=1,
        keepdims=True), 1e-12)
    D = 1 - V @ V.T; return
        float(D[np.triu_indices(len(rec_ids), 1)].mean())
def novelty(rec_ids, pop, n_users):
    return float(np.mean([-math.log2(max(1, pop.get(i, 1))
        / n_users) for i in rec_ids])) if rec_ids else 0.0
def calibration_genre(hist_ids, rec_ids):
    hp = sum(GVEC[i] for i in hist_ids); rp = sum(GVEC[i]
        for i in rec_ids)
    if np.sum(hp) == 0 or np.sum(rp) == 0: return np.nan
    return float(1 - 0.5*np.abs(hp/hp.sum() -
        rp/rp.sum()).sum())
def personalization(lists, max_pairs=20000, seed=SEED):
    L = [set(x) for x in lists]; pairs =
        list(itertools.combinations(range(len(L)), 2))
    if not pairs: return np.nan
    if len(pairs) > max_pairs: pairs =
        random.Random(seed).sample(pairs, max_pairs)
    return float(1 - np.mean([len(L[a] & L[b]) /
        max(len(L[a]), 1) for a, b in pairs]))
```

อธิบายโค้ด

ระดับรายการแนะนำ

- ILD: 1 − cosine ของแนวภายใน Top-10
- Novelty ใช้ความนิยมจาก **ชุดฝึก** เท่านั้น (ไม่รั่วจากชุดทดสอบ)
- Calibration: แนวในประวัติ vs แนวใน Top-10
- Personalization สุ่มคู่ผู้ใช้ไม่เกิน 20,000 คู่ เพื่อความเร็ว

Part 2: evaluate_ranking (1/3)

```python
def eval_users(test_df, train_ts, n=N_EVAL_USERS, seed=SEED):
    """เลือกผู้ใช้ที่ประเมินได้: อยู่ในชุดฝึกและมีรายการ relevant
ในชุดทดสอบ (สุ่ม n คนด้วย seed เดียวกันทุกโมเดล)"""
    rel = test_df[test_df.rating >= LIKE_T]
    cand = sorted(set(rel.user_id) &
        set(train_ts._raw2inner_id_users), key=int)
    if n and len(cand) > n: cand =
        sorted(random.Random(seed).sample(cand, n), key=int)
    return cand

def evaluate_ranking(model, train_ts, test_df, users_eval, k=K,
    n_neg=N_NEG, seed=SEED, keep_lists=False):
    rng = np.random.default_rng(seed)
    inner2raw = np.array([train_ts.to_raw_iid(j) for j in
        range(train_ts.n_items)])
    pop = {train_ts.to_raw_iid(i): len(v) for i, v in
        train_ts.ir.items()}
    rel_by_user = test_df[test_df.rating >=
        LIKE_T].groupby("user_id")["item_id"].apply(set).to_dict()
    rows, all_recs, lists = [], [], {}
    for uid in users_eval:
        u = train_ts.to_inner_uid(uid)
        seen = np.zeros(train_ts.n_items, bool); seen[[j for j, _ in
            train_ts.ur[u]]] = True
        cand = np.where(~seen)[0]
        rel = {r for r in rel_by_user.get(uid, set()) if r in
            train_ts._raw2inner_id_items}
        if not rel: continue
```

อธิบายโค้ด

ผู้ใช้ชุดเดียวกัน • ไม่รั่ว

- `eval_users`: ผู้ใช้ที่อยู่ในชุดฝึกและมี relevant ในชุดทดสอบ สุ่ม 300 คนด้วย seed เดียว
- `inner2raw`: แปลง inner id ของ Surprise กลับเป็นรหัสภาพยนตร์
- `seen`: เรื่องที่ผู้ใช้ให้คะแนนแล้วในชุดฝึก ⟶ ไม่นำมาแนะนำซ้ำ
- relevant ที่ไม่อยู่ในชุดฝึก (cold-start item) ถูกตัดออก

Part 2: evaluate ranking (2/3)

```
        sc_all = model.scores(u); sc = sc_all[cand]
        order = cand[np.argsort(-sc, kind="stable")]
        ranked = list(inner2raw[order]); topk =
            ranked[:k]
        y = np.isin(inner2raw[cand],
            list(rel)).astype(int)
        # Sampled protocol: รายการ relevant ทุกตัว +
รายการลบที่สุ่ม n_neg ตัว
        neg = cand[y == 0]; neg_s = rng.choice(neg,
            size=min(n_neg, len(neg)), replace=False)
        pos = cand[y == 1]; sub =
            np.concatenate([pos, neg_s])
        sub_rank = list(inner2raw[sub[np.argsort(-sc_a
            ll[sub], kind="stable")]])
```

อธิบายโค้ด

ผู้สมัครสองแบบ

- Full Catalog: ทุกเรื่องที่ยังไม่เคยดู เรียงด้วย argsort แบบ stable
- y = ป้าย relevant ของผู้สมัคร ใช้คำนวณ AUC
- Sampled: relevant ทั้งหมด + สุ่มรายการลบ 100 เรื่อง แล้วเรียงใหม่
- ใช้คะแนนชุดเดียวกัน → ต่างกันที่ protocol เท่านั้น

Part 2: evaluate_ranking (3/3)

```
        hist_ids = [train_ts.to_raw_iid(j) for j, _ in train_ts.ur[u]]
        rows.append({"user_id": uid, "n_rel": len(rel),
            f"Precision@{k}": precision_at_k(topk, rel, k),
                f"Recall@{k}": recall_at_k(topk, rel, k),
            f"F1@{k}": f1_at_k(topk, rel, k), f"HR@{k}": hr_at_k(topk,
                rel, k),
            "MRR": reciprocal_rank(ranked, rel), f"MAP@{k}":
                ap_at_k(topk, rel, k), f"nDCG@{k}": ndcg_at_k(topk,
                rel, k),
            "AUC": roc_auc_score(y, sc) if 0 < y.sum() < len(y) else
                np.nan,
            f"HR@{k}(sampled)": hr_at_k(sub_rank[:k], rel, k),
                f"nDCG@{k}(sampled)": ndcg_at_k(sub_rank[:k], rel, k),
            "ILD": ild(topk), "Novelty": novelty(topk, pop,
                train_ts.n_users),
            "Calibration": calibration_genre(hist_ids, topk), "ARP":
                float(np.mean([pop.get(i, 0) for i in topk]))})
        all_recs += topk; lists[uid] = topk
    detail = pd.DataFrame(rows)
    summary = detail.drop(columns=["user_id"]).mean(numeric_only=True).t
        o_dict()
    catalog = list(inner2raw)
    summary.update({"Coverage": coverage(all_recs, len(catalog)),
        "Gini": gini_coefficient(all_recs, catalog),
                    "Personalization":
                        personalization(list(lists.values())),
                        "n_users": len(detail)})
    return (pd.Series(summary), detail, lists) if keep_lists else
        (pd.Series(summary), detail)
```

อธิบายโค้ด

หนึ่งแถวต่อผู้ใช้

- 8 metric ความแม่น + HR/nDCG แบบ sampled + ILD, Novelty, Calibration, ARP
- สรุประดับระบบ: Coverage, Gini, Personalization และจำนวนผู้ใช้ที่ประเมิน
- `keep_lists=True` คืน Top-K ของทุกผู้ใช้ ⟶ ใช้ใน Part 9 และ GUI
- `detail` รายผู้ใช้ใช้ทำ paired bootstrap

Part 3: 6 วิธีเลือกกลุ่มตัวอย่างสำหรับ Offline Evaluation

Split	วิธีสุ่ม	ตอบคำถาม	ข้อควรระวัง
S1 Random 80/20	สุ่มระดับคะแนน	เติมช่องว่างแบบสุ่ม	ใช้อนาคตทำนายอดีต
S2 Per-user 80/20	สุ่ม 20% ของทุกผู้ใช้	ทุกผู้ใช้มีน้ำหนัก	ยังไม่สนใจเวลา
S3 Global Temporal	ตัดตามเวลา 80% แรก	ฝึกอดีต ทดสอบอนาคต	ผู้ใช้/รายการใหม่ (cold start)
S4 Per-user Temporal	20% ล่าสุดของแต่ละคน	ความชอบถัดไปของผู้ใช้เดิม	**Validation หลักของ Lab**
S5 Leave-One-Liked-Out	กันเรื่องที่ชอบ 1 เรื่อง/คน	HR@K แบบบทที่ 3–5	relevant มีเพียง 1
S6 5-Fold CV	`cross_validate`	ความแปรปรวนของ RMSE	ใช้กับ rating เท่านั้น

วิธีแบ่งข้อมูลเปลี่ยนคำถามที่การทดลองตอบ → ต้องระบุใน Protocol Card ทุกครั้ง

Part 3: ฟังก์ชันแบ่งข้อมูล 5 แบบ

```python
def split_random(df, test_size=0.2, seed=SEED):
    te = df.sample(frac=test_size, random_state=seed); return
        df.drop(te.index), te

def split_per_user(df, test_size=0.2, seed=SEED):
    te = df.groupby("user_id").sample(frac=test_size,
        random_state=seed)
    return df.drop(te.index), te

def split_global_temporal(df, train_frac=0.8):
    cut = df.timestamp.quantile(train_frac); return
        df[df.timestamp <= cut], df[df.timestamp > cut]

def split_user_temporal(df, test_size=0.2):
    d = df.sort_values(["user_id", "timestamp"]).copy()
    d["rk"] = d.groupby("user_id").cumcount(); d["n"] =
        d.groupby("user_id")["item_id"].transform("size")
    is_test = d.rk >= np.ceil(d.n * (1 - test_size))
    return df.loc[d.index[~is_test]], df.loc[d.index[is_test]]

def split_leave_one_liked_out(df, seed=SEED):
    liked = df[df.rating >= LIKE_T]
    te = liked.groupby("user_id").sample(n=1,
        random_state=seed)
    return df.drop(te.index), te
```

อธิบายโค้ด

ทุกฟังก์ชันคืน (train_df, test_df)

- `split_per_user`: `groupby().sample(frac)` → stratified ต่อผู้ใช้
- `split_global_temporal`: ตัดที่ควอนไทล์ 0.8 ของ timestamp
- `split_user_temporal`: เรียงตามเวลา แล้วให้ 20% ท้ายของแต่ละคนเป็น test
- `split_leave_one_liked_out`: สุ่มเรื่องที่ชอบ 1 เรื่องต่อผู้ใช้

Part 3: ตารางเปรียบเทียบ Split

```
SPLITS = {"S1 Random 80/20": split_random(ratings), "S2
    Per-user 80/20": split_per_user(ratings),
          "S3 Global Temporal":
              split_global_temporal(ratings), "S4 Per-user
              Temporal": split_user_temporal(ratings),
          "S5 Leave-One-Liked-Out":
              split_leave_one_liked_out(ratings)}
MAIN_SPLIT = "S4 Per-user Temporal"

split_rows = []
for name, (tr, te) in SPLITS.items():
    split_rows.append({"Split": name, "train": len(tr),
        "test": len(te), "test users": te.user_id.nunique(),
        "cold-start users": len(set(te.user_id) -
            set(tr.user_id)), "cold-start items":
            len(set(te.item_id) - set(tr.item_id)),
        "test liked %": 100 * (te.rating >= LIKE_T).mean(),
        "train end": pd.to_datetime(tr.timestamp.max(),
            unit="s").date(),
        "test start": pd.to_datetime(te.timestamp.min(),
            unit="s").date()})
split_table = pd.DataFrame(split_rows);
    display(split_table.round(2))
```

อธิบายโค้ด

ตรวจคุณสมบัติของแต่ละ Split

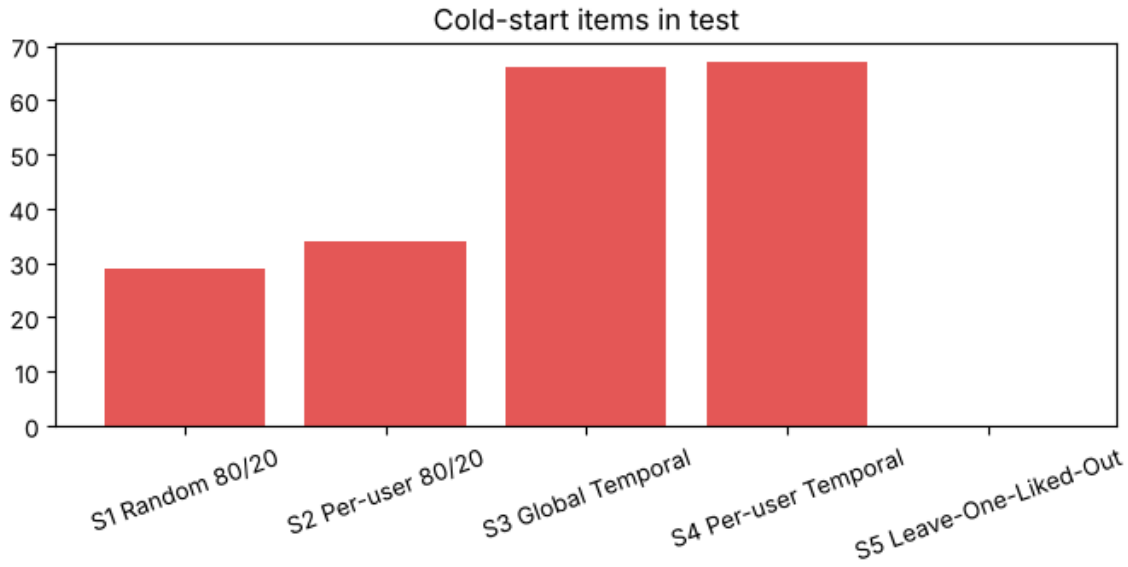
- ขนาด train/test และจำนวนผู้ใช้ในชุดทดสอบ
- cold-start users/items: อยู่ใน test แต่ไม่อยู่ใน train
- สัดส่วนที่ชอบใน test และช่วงเวลาสิ้นสุด train / เริ่ม test
- `MAIN_SPLIT = S4` ใช้เป็น Validation หลัก

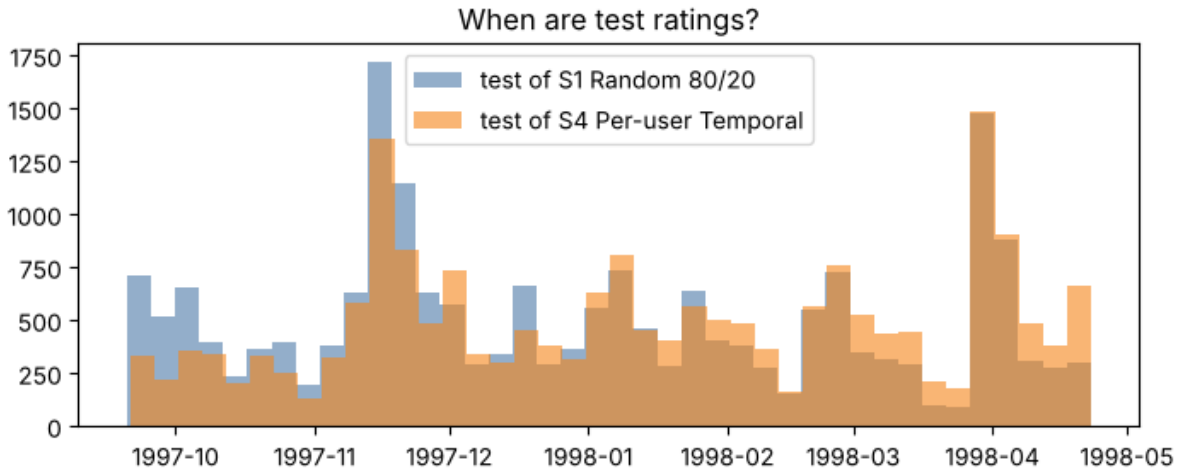
Part 3: ผลการแบ่งข้อมูล (จาก Notebook)

Split	train	test	test users	cold users	cold items	liked %
S1 Random 80/20	80,000	20,000	940	0	29	54.95
S2 Per-user 80/20	80,000	20,000	943	0	34	54.79
S3 Global Temporal	80,003	19,997	301	192	66	56.52
S4 Per-user Temporal	80,367	19,633	943	0	67	47.77
S5 Leave-One-Liked-Out	99,058	942	942	0	0	100.00

S3 มีผู้ใช้ใหม่ 192 คนในชุดทดสอบ ⟶ การกรองร่วมทำนายไม่ได้ (5.15.1) • S4 ทุกคนอยู่ในชุดฝึก

Part 3: Cold start และช่วงเวลาของชุดทดสอบ





- ซ้าย: จำนวนรายการใหม่ในชุดทดสอบของแต่ละ Split — Split ตามเวลามีมากกว่า
- ขวา: Random ทดสอบกระจายทุกช่วงเวลา แต่ Per-user Temporal เอียงไปช่วงท้าย ⟶ ใกล้สถานการณ์ใช้งานจริง

Part 4: อัลกอริทึมที่เปรียบเทียบ

ตระกูล (บทที่ 5)	อัลกอริทึม	ไฮเพอร์พารามิเตอร์	หัวข้อ
Baseline	NormalPredictor, BaselineOnly (ALS), Popularity*	bsl: als 10 รอบ	5.8
Memory-based	KNNBasic user cos • KNNWithMeans user/item pearson • KNNWithZScore	k = 40	5.4–5.6
Memory-based	KNNBaseline item pearson_baseline	k = 40, shrinkage = 100	5.4.3, 5.6
Latent factor	SVD • SVD++ • NMF	f = 100 / 20 / 15	5.9–5.10
อื่น ๆ	SlopeOne • CoClustering	3×3 clusters	—
Implicit ranking	EASE*	λ = 500	5.12.2

* เขียนเองใน Core Functions เพราะ Surprise ไม่มี

Part 4: make_models (1/2)

```python
def make_models():
    sim = lambda name, ub, **kw: {"name": name, "user_based": ub,
        **kw}
    return [
        SurpriseModel("NormalPredictor", lambda:
            NormalPredictor(), "Baseline"),
        SurpriseModel("BaselineOnly", lambda:
            BaselineOnly(bsl_options={"method": "als",
            "n_epochs": 10}, verbose=False), "Baseline"),
        PopularityModel(),
        SurpriseModel("UserKNN-Basic(cos)", lambda:
            KNNBasic(k=40, sim_options=sim("cosine", True),
            verbose=False), "Memory-based"),
        SurpriseModel("UserKNN-Means(pearson)", lambda:
            KNNWithMeans(k=40, sim_options=sim("pearson", True),
            verbose=False), "Memory-based"),
        SurpriseModel("ItemKNN-Means(pearson)", lambda:
            KNNWithMeans(k=40, sim_options=sim("pearson", False),
            verbose=False), "Memory-based"),
        SurpriseModel("UserKNN-ZScore", lambda:
            KNNWithZScore(k=40, sim_options=sim("pearson", True),
            verbose=False), "Memory-based"),
        SurpriseModel("ItemKNN-Baseline", lambda:
            KNNBaseline(k=40, sim_options=sim("pearson_baseline",
            False, shrinkage=100), verbose=False),
            "Memory-based"),
```

อธิบายโค้ด

Baseline + Memory-based

- `sim(...)` สร้าง `sim_options` สั้น ๆ: ชื่อความคล้าย + user/item based
- ห่อด้วย `lambda` ⟶ สร้างอัลกอริทึมใหม่ทุกครั้งที่ fit (ไม่มีสถานะค้าง)
- kNN 5 แบบต่างกันที่ความคล้าย (cosine/pearson/pearson_baseline) และการปรับสเกล (mean/z-score/baseline)
- KNNBaseline ใช้ `shrinkage=100` ลดความคล้ายจากหลักฐานน้อย (5.4.3)

Part 4: make_models (2/2)

```
        SurpriseModel("SVD", lambda:
            SVD(n_factors=100, n_epochs=20,
            random_state=SEED), "Latent factor"),
        SurpriseModel("SVD++", lambda:
            SVDpp(n_factors=20, n_epochs=20,
            random_state=SEED), "Latent factor"),
        SurpriseModel("NMF", lambda:
            NMF(n_factors=15, n_epochs=50,
            random_state=SEED), "Latent factor"),
        SurpriseModel("SlopeOne", lambda: SlopeOne(),
            "Other"),
        SurpriseModel("CoClustering", lambda:
            CoClustering(n_cltr_u=3, n_cltr_i=3,
            random_state=SEED), "Other"),
        EASEModel(lam=500.0)]

MODEL_NAMES = [m.name for m in make_models()]
RATING_MODELS = [m.name for m in make_models() if
    m.rating_model]
FAMILY = {m.name: m.family for m in make_models()}
def get_model(name): return [m for m in make_models()
    if m.name == name][0]
```

อธิบายโค้ด

Latent factor + อื่น ๆ + registry

- SVD (Funk-SVD + bias, SGD) f = 100 • SVD++ f = 20 • NMF f = 15
- `random_state=SEED` → ผลคงที่ทุกครั้งที่รัน
- `RATING_MODELS` = 12 ตัวที่ทำนายคะแนนได้ (ไม่รวม Popularity, EASE)
- เพิ่มอัลกอริทึม = เพิ่มหนึ่งบรรทัดในรายการ

Part 5: Exp.1 Rating บน 4 Split

```
TRAINSETS = {name: (df_to_trainset(tr), te) for name, (tr,
    te) in SPLITS.items()}
rating_rows, FITTED = [], {}
for split_name in ["S1 Random 80/20", "S2 Per-user 80/20",
    "S3 Global Temporal", "S4 Per-user Temporal"]:
    ts_, te = TRAINSETS[split_name]; testset =
        df_to_testset(te)
    for name in RATING_MODELS:
        t0 = time.time(); m = get_model(name).fit(ts_); fit_t
            = time.time() - t0
        t0 = time.time(); preds = m.algo.test(testset);
            test_t = time.time() - t0
        rating_rows.append({"Split": split_name, "Model":
            name, "Family": FAMILY[name],
            **rating_metrics(preds),
                           "fit_s": fit_t, "test_s": test_t})
        if split_name == MAIN_SPLIT: FITTED[name] = m
    print("done", split_name)
rating_results = pd.DataFrame(rating_rows)
display(rating_results.pivot(index="Model", columns="Split",
    values="RMSE").sort_values(MAIN_SPLIT).round(4))
display(rating_results[rating_results.Split ==
    MAIN_SPLIT].sort_values("RMSE").round(4).reset_index(drop=
    True))
```

อธิบายโค้ด

ฝึกบน train ทำนาย test

- **TRAINSETS** แปลงทุก split เป็น trainset ของ Surprise ครั้งเดียว
- วัด RMSE, MAE, FCP, Impossible% และเวลา fit/test
- เก็บโมเดลของ S4 ไว้ใน **FITTED** ⟶ ใช้ต่อใน Part 6 และ GUI
- **pivot** เป็นตาราง Model × Split

Part 5: 5-Fold Cross-Validation (S6)

```
data_all = Dataset.load_from_df(ratings[["user_id",
    "item_id", "rating"]], READER)
cv_rows = []
for name in RATING_MODELS:
    r = cross_validate(get_model(name).factory(),
        data_all, measures=["RMSE", "MAE"], cv=5,
        verbose=False)
    cv_rows.append({"Model": name, "RMSE mean":
        np.mean(r["test_rmse"]), "RMSE std":
        np.std(r["test_rmse"]),
                    "MAE mean":
                        np.mean(r["test_mae"]), "MAE
                        std": np.std(r["test_mae"]),
                        "fit_s": np.mean(r["fit_time"]
                        )})
cv_results = pd.DataFrame(cv_rows).sort_values("RMSE
    mean").reset_index(drop=True)
print("S6 — 5-Fold CV"); display(cv_results.round(4))
```

อธิบายโค้ด

ค่าเฉลี่ย ± ส่วนเบี่ยงเบนมาตรฐาน

- `cross_validate` ของ Surprise แบ่ง 5 fold แบบสุ่มให้อัตโนมัติ
- รายงาน mean ± std ของ RMSE/MAE และเวลา fit
- std เล็ก (≈ 0.003–0.007) ⟶ ต่างกัน 0.01 ขึ้นไปจึงน่าเชื่อ

Part 5: ผล Exp.1 — RMSE บน 4 Split

อัลกอริทึม	S1 Random	S2 Per-user	S3 Global Temp.	S4 Per-user Temp.
SVD++	0.9186	0.9254	1.0251	**0.9643**
SVD	0.9383	0.9411	1.0287	0.9839
ItemKNN-Baseline	**0.9156**	**0.9181**	1.0295	0.9856
SlopeOne	0.9447	0.9508	1.1023	0.9927
BaselineOnly	0.9417	0.9495	1.0300	0.9985
ItemKNN-Means	0.9426	0.9482	1.1014	1.0010
UserKNN-Means	0.9469	0.9585	1.1085	1.0058
NMF	0.9632	0.9691	1.1043	1.0182
UserKNN-Basic(cos)	1.0121	1.0271	1.1027	1.0699
NormalPredictor	1.5225	1.5245	1.5215	1.5588

Split ตามเวลาให้ RMSE สูงกว่าแบบสุ่มเสมอ — Random split ให้ภาพที่ดีเกินจริง

Part 5: ผล 5-Fold CV (S6)

อัลกอริทึม	RMSE mean	RMSE std	MAE mean	fit (s)
ItemKNN-Baseline	0.9164	0.0072	0.7182	0.34
SVD++	0.9180	0.0050	0.7208	11.34
SVD	0.9365	0.0033	0.7381	0.45
ItemKNN-Means(pearson)	0.9415	0.0043	0.7384	0.37
SlopeOne	0.9440	0.0073	0.7420	0.29
BaselineOnly	0.9442	0.0031	0.7484	0.07
UserKNN-Means(pearson)	0.9494	0.0052	0.7431	0.23
NMF	0.9636	0.0018	0.7574	0.73
NormalPredictor	1.5192	0.0043	1.2189	0.05

บนข้อมูลจริง โมเดลที่เรียนพารามิเตอร์ชนะค่าเฉลี่ย — ต่างจาก MovieMini-12 ใน LAB 1 • SVD++ แม่นแต่ช้ากว่า 25 เท่า

Part 6: Exp.2 Ranking บน S4

```
ts_main, te_main = TRAINSETS[MAIN_SPLIT]
USERS_EVAL = eval_users(te_main, ts_main)
print(f"ประเมิน {len(USERS_EVAL)} ผู้ใช้จาก
    {te_main.user_id.nunique()} คนในชุดทดสอบ")
for name in MODEL_NAMES:
    if name not in FITTED: FITTED[name] =
        get_model(name).fit(ts_main)

rank_rows, RANK_DETAIL, RANK_LISTS = [], {}, {}
for name in MODEL_NAMES:
    t0 = time.time()
    s, d, lists = evaluate_ranking(FITTED[name],
        ts_main, te_main, USERS_EVAL, keep_lists=True)
    rank_rows.append({"Model": name, "Family":
        FAMILY[name], **s.to_dict(), "rank_s":
        time.time() - t0})
    RANK_DETAIL[name], RANK_LISTS[name] = d, lists
    print(f"{name:24s} nDCG@{K}={s[f'nDCG@{K}']:.4f}
        ({time.time()-t0:.1f}s)")
```

อธิบายโค้ด

ประเมินผู้ใช้ 300 คนชุดเดียวกัน

- USERS_EVAL สุ่มครั้งเดียว ทุกโมเดลใช้ผู้ใช้ชุดเดียวกัน
- ใช้โมเดลที่ฝึกแล้วใน FITTED (ฝึกเพิ่มเฉพาะ Popularity/EASE)
- เก็บ RANK_DETAIL รายผู้ใช้ และ RANK_LISTS Top-10
- ใช้เวลา: SVD/EASE ≈ 1 วินาที, kNN ≈ 20 วินาที (เรียก estimate ทีละเรื่อง)

Part 6: ผล Exp.2 — Ranking @10 (Full Catalog, 300 ผู้ใช้)

อัลกอริทึม	P@10	R@10	HR@10	MRR	MAP@10	nDCG@10	AUC
EASE	0.1360	0.1725	0.6433	0.3474	0.1056	**0.1912**	0.8918
Popularity	0.0713	0.0743	0.4033	0.2196	0.0448	0.0938	0.8237
SVD++	0.0510	0.0461	0.3200	0.1405	0.0228	0.0576	0.6564
SVD	0.0423	0.0393	0.2800	0.1196	0.0190	0.0481	0.6435
BaselineOnly	0.0427	0.0411	0.2667	0.0907	0.0170	0.0437	0.6342
ItemKNN-Baseline	0.0267	0.0191	0.1833	0.0810	0.0107	0.0283	0.6532
ItemKNN-Means	0.0017	0.0008	0.0133	0.0164	0.0002	0.0012	0.6838
SlopeOne	0.0000	0.0000	0.0000	0.0127	0.0000	0.0000	0.6636

RMSE ดีไม่ได้แปลว่าจัดอันดับดี: kNN/SlopeOne ดันเรื่องที่มีคะแนนน้อยแต่ค่าทำนาย ≈ 5 ขึ้นบนสุด (5.4.3, 5.17.2)

Part 7: ผล Exp.3 — Beyond-Accuracy (Top-10)

อัลกอริทึม	Coverage	Gini	ILD	Novelty	Calibration	Personal.	ARP
EASE	0.138	0.937	0.703	1.95	**0.685**	0.906	265
Popularity	0.042	0.987	0.692	1.39	0.638	0.561	**367**
SVD++	0.121	0.969	0.741	2.89	0.590	0.728	168
SVD	0.119	0.968	0.751	2.74	0.593	0.734	169
ItemKNN-Baseline	**0.270**	0.902	0.741	5.16	0.573	0.906	93
UserKNN-Means	0.080	0.979	0.653	8.53	0.490	0.670	3
SlopeOne	0.050	0.988	0.529	8.87	0.415	0.472	2
NormalPredictor	0.846	0.397	0.770	5.92	0.596	0.994	44

Popularity: Coverage ต่ำ ARP สูง • kNN: Novelty สูงแต่ไม่แม่น • สุ่ม: Coverage สูงสุดแต่ไร้ประโยชน์ ⟶ อ่านคู่กับ Accuracy เสมอ

Part 8: Exp.4 ผลของ Protocol

```
REP_MODELS = ["Popularity", "BaselineOnly",
    "ItemKNN-Baseline", "SVD", "EASE"]
proto_rows = []
for split_name in ["S1 Random 80/20", "S4 Per-user
    Temporal", "S5 Leave-One-Liked-Out"]:
    ts_, te = TRAINSETS[split_name]; ue = eval_users(te,
        ts_)
    for name in REP_MODELS:
        m = FITTED[name] if split_name == MAIN_SPLIT else
            get_model(name).fit(ts_)
        s, _ = evaluate_ranking(m, ts_, te, ue)
        proto_rows.append({"Split": split_name, "Model":
            name, f"HR@{K}": s[f"HR@{K}"], f"nDCG@{K}":
            s[f"nDCG@{K}"],
                           f"HR@{K}(sampled)":
                               s[f"HR@{K}(sampled)"],
                               f"nDCG@{K}(sampled)":
                               s[f"nDCG@{K}(sampled)"],
                               "MRR": s["MRR"]})
protocol_results = pd.DataFrame(proto_rows)
display(protocol_results.round(4))
display(protocol_results.pivot(index="Model",
    columns="Split", values=f"nDCG@{K}").round(4))
```

อธิบายโค้ด

Split × Candidate protocol

- โมเดลตัวแทน 5 ตัว × 3 Split (S1, S4, S5)
- แต่ละ split สุ่มผู้ใช้ประเมินใหม่ แต่ใช้ seed เดียวกัน
- รายงานทั้ง Full Catalog และ Sampled 100 negatives
- ใช้ซ้ำ **FITTED** เมื่อเป็น S4 ⟶ ไม่ต้องฝึกใหม่

Part 8: ผล Exp.4 — nDCG@10 ตาม Split และ Candidate

อัลกอริทึม	S1 Full	S1 Sampled	S4 Full	S4 Sampled	S5 Full	S5 Sampled
EASE	0.3502	0.7515	0.1912	0.5989	0.2568	0.6098
Popularity	0.1829	0.5567	0.0938	0.4140	0.0797	0.3632
SVD	0.0812	0.3555	0.0481	0.2623	0.0327	0.2142
BaselineOnly	0.0813	0.3695	0.0437	0.2569	0.0321	0.2149
ItemKNN-Baseline	0.0584	0.3376	0.0283	0.2216	0.0229	0.2158

Sampled สูงกว่า Full 2–9 เท่า และ Random (S1) สูงกว่า Temporal (S4) ~2 เท่า • S5 Sampled: ItemKNN แซง SVD และ BaselineOnly

Part 9: Validation — ค่าทำนายเทียบค่าจริง

```
SAMPLE_USERS = (te_main[te_main.rating >=
    LIKE_T].groupby("user_id").size()
                .loc[lambda s:
                    s.index.isin(USERS_EVAL)].sort_values(ascending=Fals
                    e).index.tolist())
SAMPLE_USERS = [SAMPLE_USERS[p] for p in (0, len(SAMPLE_USERS)//4,
    len(SAMPLE_USERS)//2)]
print("ผู้ใช้ตัวอย่าง:", SAMPLE_USERS)

def short(iid, n=28):
    t = TITLE.get(iid, iid); return t if len(t) <= n else t[:n-1] + "…"

# 9.1 Rating
for uid in SAMPLE_USERS[:2]:
    info = users.set_index("user_id").loc[uid]
    print(f"\nผู้ใช้ {uid}: อายุ {info.age} เพศ {info.gender} อาชีพ
        {info.occupation}")
    sub = te_main[te_main.user_id ==
        uid].sort_values("timestamp").head(8)
    tab = pd.DataFrame({"movie": [short(i) for i in sub.item_id],
        "true": sub.rating.values})
    for name in RATING_MODELS:
        tab[name] = [FITTED[name].predict(uid, i) for i in sub.item_id]
    display(tab.round(2))
    print("MAE ต่อโมเดล (เฉพาะแถวด้านบน):", {n:
        round(float(np.mean(np.abs(tab[n] - tab.true))), 3) for n in
        RATING_MODELS})
```

อธิบายโค้ด

เลือกผู้ใช้ตัวอย่างอัตโนมัติ

- เรียงผู้ใช้ตามจำนวน relevant ใน Validation แล้วเลือกตำแหน่ง 0, ¼, ½ ⟶ ได้ 130, 56, 767
- แสดงอายุ เพศ อาชีพ ก่อนตาราง
- 8 คะแนนแรกตามเวลาในชุด Validation: ค่าจริง vs ค่าทำนายของ 12 อัลกอริทึม
- พิมพ์ MAE ต่อโมเดลของแถวที่แสดง

Part 9: Validation — Top-10 ทุกอัลกอริทึม

```
# 9.2 Ranking
for uid in SAMPLE_USERS:
    rel = set(te_main[(te_main.user_id == uid) &
        (te_main.rating >= LIKE_T)].item_id)
    comp = pd.DataFrame({name: [("✓ " if i in rel
        else "") + short(i) for i in
        RANK_LISTS[name][uid]] for name in
        MODEL_NAMES},
                        index=[f"#{n}" for n in
                            range(1, K+1)]).T
    comp["hits"] = [sum(i in rel for i in
        RANK_LISTS[name][uid]) for name in
        MODEL_NAMES]
    print(f"\nผู้ใช้ {uid}: relevant ใน Validation =
        {len(rel)} เรื่อง")
    display(comp)
```

อธิบายโค้ด

✓ = ผู้ใช้ชอบเรื่องนั้นจริงในอนาคต

- ใช้ `RANK_LISTS` จาก Part 6 (ไม่ต้องคำนวณใหม่)
- ชื่อเรื่องตัดให้สั้นด้วย `short()`
- คอลัมน์ `hits` = จำนวนเรื่องที่แนะนำถูกใน Top-10

Part 9: ตัวอย่าง Output (Rating) — ผู้ใช้ 130 (20 ปี, M)

ภาพยนตร์	จริง	Baseline	UserKNN	ItemKNN-B	SVD	SVD++	SlopeOne
Devil's Advocate (1997)	4	4.12	3.84	4.04	3.78	4.10	4.23
Gattaca (1997)	5	4.27	4.09	4.41	4.62	4.67	4.49
Gang Related (1997)	4	3.55	3.20	3.55	3.54	3.11	2.77
Original Gangstas (1996)	3	3.88	3.68	3.83	3.72	3.93	3.30
Leaving Las Vegas (1995)	4	4.30	3.88	4.12	3.80	3.63	4.43
Super Mario Bros. (1993)	3	3.31	2.89	3.59	3.33	3.41	2.84
Clear and Present Danger	4	4.25	3.85	4.32	4.07	4.15	4.33

ทุกอัลกอริทึม (ยกเว้นสุ่ม) ทำนายใกล้ค่าจริงในระดับ ±0.5 — ความต่างเห็นชัดใน Ranking มากกว่า

Part 9: ตัวอย่าง Output (Ranking) — ผู้ใช้ 56 (25 ปี, librarian)

อัลกอริทึม	#1	#2	#3	hits@10
EASE	✓ Die Hard (1988)	✓ Speed (1994)	✓ Top Gun (1986)	4
SVD	Schindler's List	✓ Die Hard (1988)	Secrets & Lies	1
SVD++	Titanic	One Flew Over the Cuckoo's...	Good Will Hunting	1
Popularity	Fargo	Godfather, The	English Patient, The	0
ItemKNN-Baseline	Santa with Muscles	Usual Suspects, The	Dangerous Beauty	0
UserKNN-Means	Aiqing wansui	City of Industry	Hearts and Minds	0

ผู้ใช้ชอบหนังแอ็กชันยุค 80–90 • EASE จับรูปแบบ "ดูด้วยกัน" ได้ • kNN แนะนำหนังหายากที่มีคะแนนน้อย

Part 10: ตารางสรุป + กราฟ 6 รูป

```
summary = ranking_results[["Model", "Family", f"HR@{K}", "MRR",
    f"nDCG@{K}", "AUC"] + BEYOND_COLS].merge(
    rating_results[rating_results.Split == MAIN_SPLIT][["Model",
        "RMSE", "MAE", "fit_s"]], on="Model", how="left")
display(summary.sort_values(f"nDCG@{K}",
    ascending=False).round(4).reset_index(drop=True))

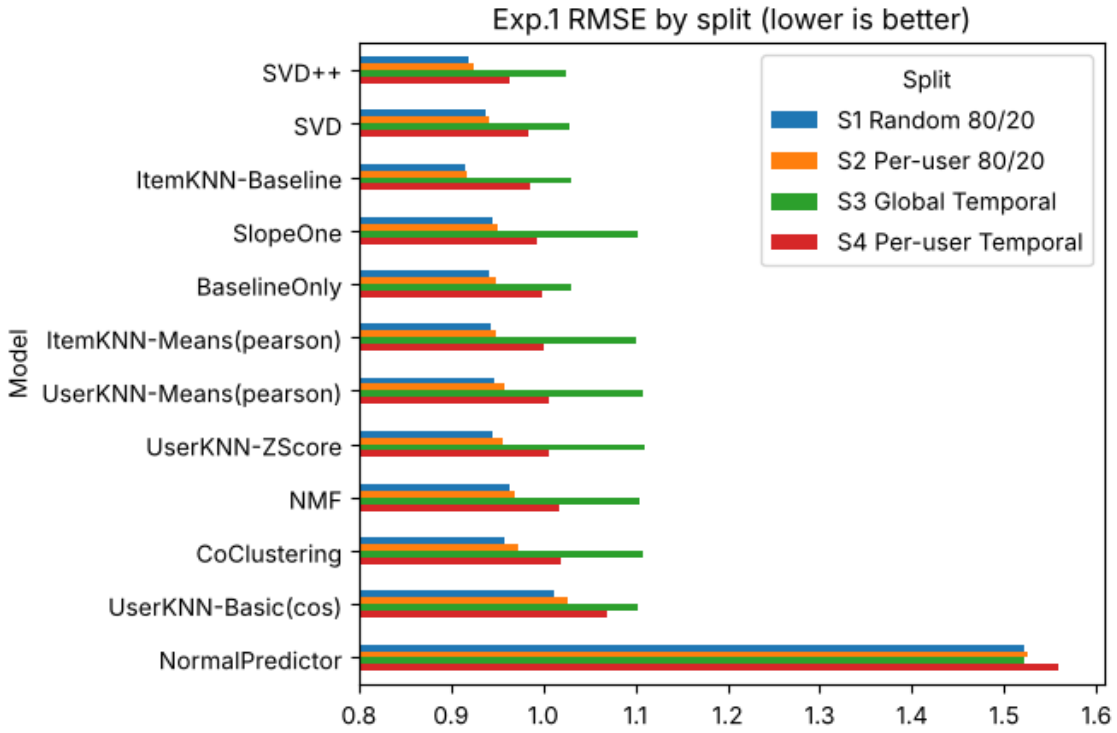
fig, axes = plt.subplots(2, 3, figsize=(20, 10))
piv = rating_results.pivot(index="Model", columns="Split",
    values="RMSE").sort_values(MAIN_SPLIT)
piv.plot.barh(ax=axes[0, 0]); axes[0, 0].set_xlim(0.8,
    piv.values[np.isfinite(piv.values)].max() + .05)
axes[0, 0].set_title("Exp.1 RMSE by split (lower is better)");
    axes[0, 0].invert_yaxis()
cv_results.set_index("Model")["RMSE mean"].plot.barh(ax=axes[0,
    1], xerr=cv_results.set_index("Model")["RMSE std"],
    color="#72B7B2")
axes[0, 1].set_xlim(0.8, cv_results["RMSE mean"].max() + .05);
    axes[0, 1].invert_yaxis(); axes[0, 1].set_title("S6 5-fold CV
    RMSE (± std)")
rk = ranking_results.set_index("Model")[[f"Precision@{K}",
    f"Recall@{K}", f"nDCG@{K}", "MRR"]].sort_values(f"nDCG@{K}")
rk.plot.barh(ax=axes[0, 2]); axes[0, 2].set_title(f"Exp.2 Ranking
    @{K} (Full catalog)")
```

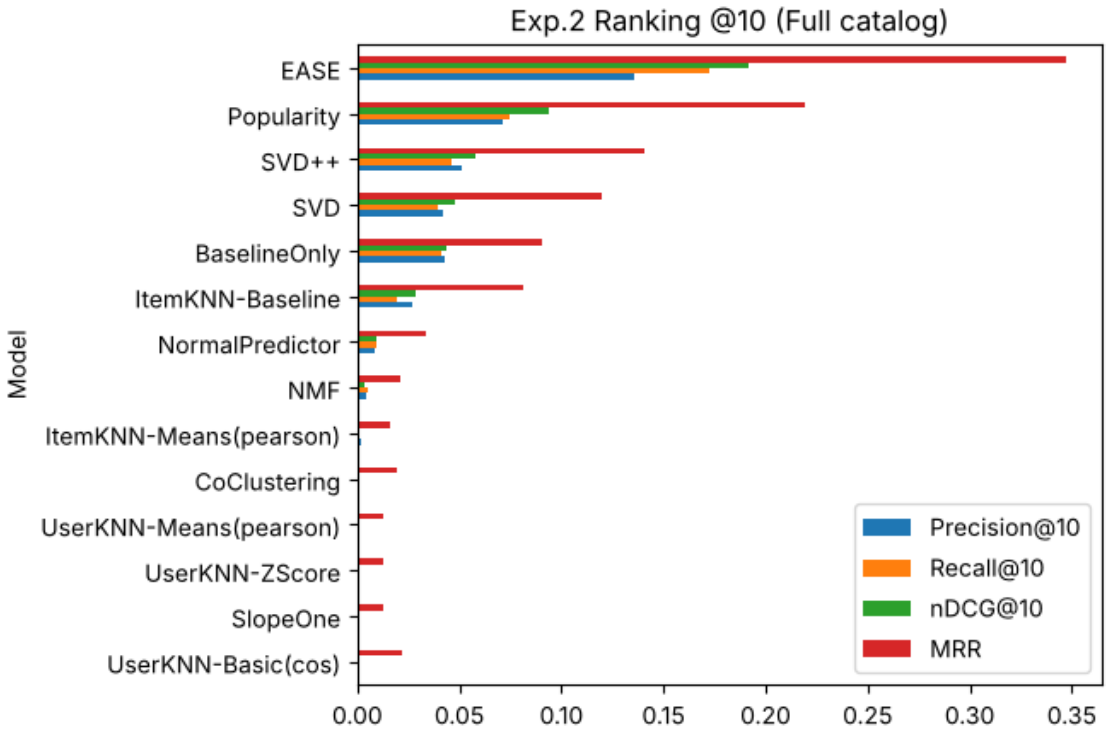
อธิบายโค้ด

รวมทุกการทดลองไว้ที่เดียว

- `summary` = ranking + beyond (S4) merge กับ RMSE/MAE/เวลา fit ของ S4
- กราฟ 2×3: RMSE ตาม split • CV ± std • ranking • beyond • trade-off • protocol
- `xlim` เริ่มที่ 0.8 เพื่อให้เห็นความต่างของ RMSE ชัด

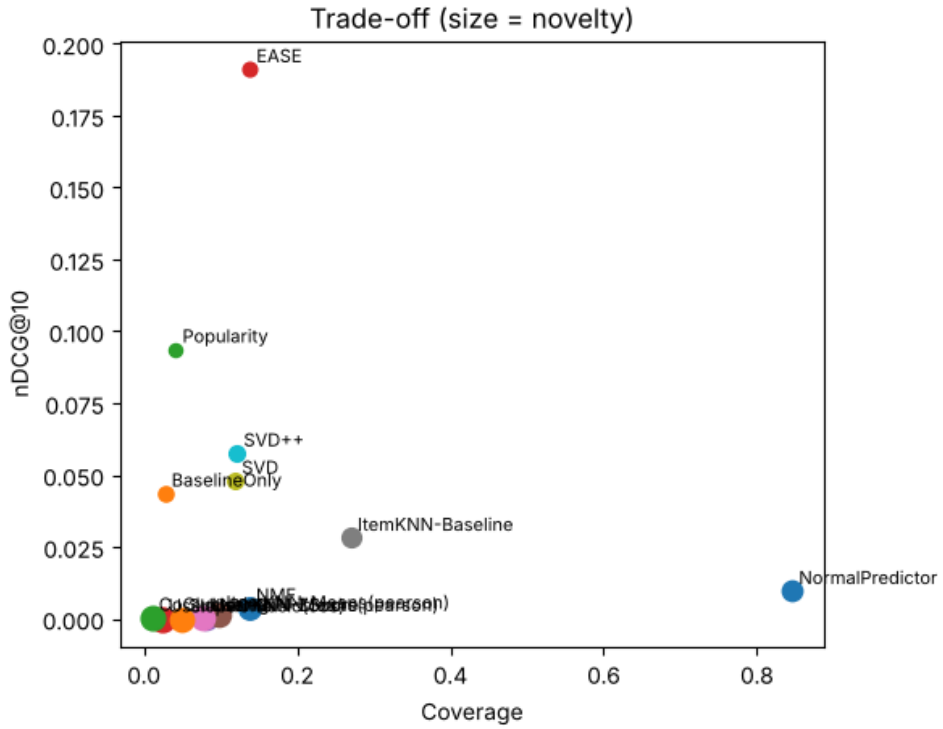
Part 10: Chart — Rating และ Ranking

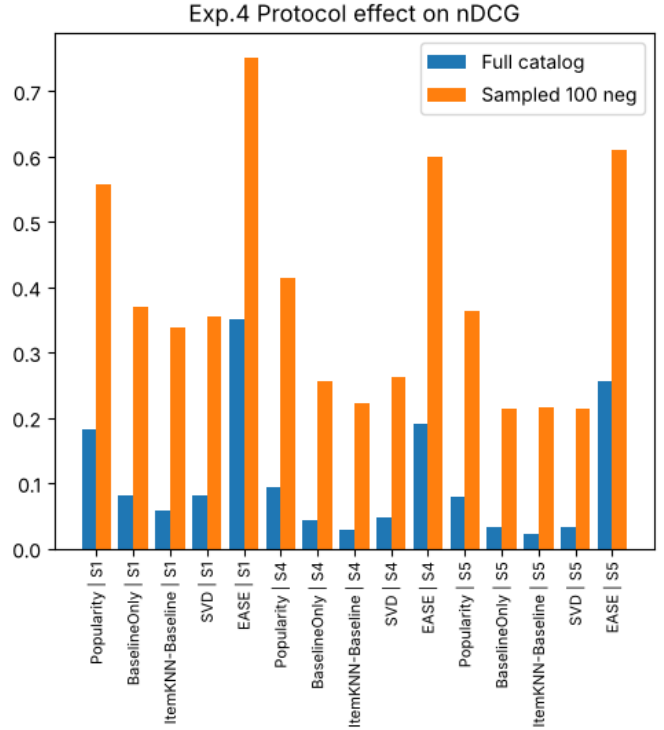




- ซ้าย: RMSE ทุก split — S3 Global Temporal (เขียว) สูงสุดเพราะ cold start • NormalPredictor แย่สุดทุก split
- ขวา: Ranking — EASE นำทุก metric, Popularity อันดับ 2 ขณะที่ kNN แทบเป็นศูนย์

Part 10: Chart — Beyond-Accuracy และ Protocol





- ซ้าย: nDCG vs Coverage — EASE แม่นสุด, ItemKNN-Baseline กระจายดีกว่าแต่แม่นน้อยกว่า
- ขวา: Sampled (ส้ม) สูงกว่า Full (น้ำเงิน) ทุกกรณี ⟶ ห้ามเทียบตัวเลขข้าม protocol

Part 10: Paired Bootstrap ระดับผู้ใช้

```
def paired_bootstrap(a, b, metric=f"nDCG@{K}", B=2000,
    seed=SEED):
    da = RANK_DETAIL[a].set_index("user_id")[metric]; db
        = RANK_DETAIL[b].set_index("user_id")[metric]
    diff = (da - db.reindex(da.index)).dropna().values;
        rng = np.random.default_rng(seed)
    boots = rng.choice(diff, size=(B, len(diff)),
        replace=True).mean(1)
    return diff.mean(), np.percentile(boots, [2.5,
        97.5]), (diff > 0).sum(), (diff == 0).sum(),
        (diff < 0).sum()
order = ranking_results.sort_values(f"nDCG@{K}",
    ascending=False).Model.tolist()
for other in list(dict.fromkeys(order[1:4] + ["SVD",
    "ItemKNN-Baseline"])):
    if other == order[0]: continue
    m_, ci, w, t, l = paired_bootstrap(order[0], other)
    print(f"{order[0]} - {other}: Δ={m_:.4f}  95%
        CI=[{ci[0]:.4f}, {ci[1]:.4f}]
        win/tie/loss={w}/{t}/{l}",
          "(สรุปไม่ได้)" if ci[0] <= 0 <= ci[1] else
              "(ต่างอย่างมีหลักฐาน)")
```

อธิบายโค้ด

ความต่างเชื่อได้แค่ไหน

- จับคู่ nDCG@10 ของผู้ใช้คนเดียวกัน แล้ว bootstrap 2,000 รอบ
- รายงาน win / tie / loss รายผู้ใช้
- EASE – Popularity: Δ 0.098, CI [0.075, 0.119]
- EASE – SVD: Δ 0.143, CI [0.121, 0.166] ⟶ ต่างอย่างมีหลักฐาน

Part 11: GUI — โมเดลตามโหมดและข้อมูลผู้ใช้

```
ts_full = df_to_trainset(ratings)
FITTED_FULL = {}
def fitted_for(mode, name):
    if mode.startswith("Validation"): return FITTED[name], ts_main
    if name not in FITTED_FULL: FITTED_FULL[name] =
        get_model(name).fit(ts_full)
    return FITTED_FULL[name], ts_full

USER_CHOICES = sorted(ratings.user_id.unique(), key=int)
MOVIE_CHOICES = [f"{i} | {TITLE[i]}" for i in sorted(items.item_id,
    key=int)]

def user_profile(uid, ts_):
    info = users.set_index("user_id").loc[uid]
    u = ts_.to_inner_uid(uid); hist = [(ts_.to_raw_iid(j), r) for j, r in
        ts_.ur[u]]
    liked = [i for i, r in hist if r >= LIKE_T]
    g = pd.Series([x for i in liked for x in
        GENRE[i].split("|")]).value_counts().head(5)
    md = (f"### ผู้ใช้ {uid}\n- อายุ **{info.age}** · เพศ **{info.gender}**
        · อาชีพ **{info.occupation}** · zip {info.zip}\n"
          f"- จำนวนคะแนน (ในข้อมูลที่ใช้ฝึก): **{len(hist)}** · ค่าเฉลี่ย
              **{np.mean([r for _, r in hist]):.2f}** · ชอบ
              **{len(liked)}**เรื่อง\n"
          f"- แนวที่ชอบมากที่สุด: {', '.join(f'{k} ({v})' for k, v in
              g.items())}")
    h = pd.DataFrame([{"Movie": TITLE[i], "Genre": GENRE[i], "Rating": r}
        for i, r in hist])
    return md, h.sort_values("Rating", ascending=False).head(15)
```

อธิบายโค้ด

สองโหมด: Validation / Full data

- `fitted_for`: โหมด Validation ใช้โมเดลที่ฝึกบน S4, โหมด Full ฝึกบนข้อมูลทั้งหมด (cache ใน `FITTED_FULL`)
- `user_profile`: อายุ เพศ อาชีพ zip + จำนวนคะแนน ค่าเฉลี่ย และแนวที่ชอบ 5 อันดับ
- ประวัติแสดง 15 เรื่องที่ให้คะแนนสูงสุด

Part 11: GUI — สร้างรายการแนะนำ

```
def recommend(uid, name, k, mode):
    m, ts_ = fitted_for(mode, name); u = ts_.to_inner_uid(uid)
    seen = {j for j, _ in ts_.ur[u]}; sc = m.scores(u)
    order = [j for j in np.argsort(-sc, kind="stable") if j
        not in seen][:int(k)]
    rel = set(te_main[(te_main.user_id == uid) &
        (te_main.rating >= LIKE_T)].item_id) if
        mode.startswith("Validation") else set()
    pop = {ts_.to_raw_iid(i): len(v) for i, v in
        ts_.ir.items()}
    return pd.DataFrame([{"Rank": n, "Movie":
        TITLE[ts_.to_raw_iid(j)], "Year":
        YEAR.get(ts_.to_raw_iid(j)),
                          "Genre": GENRE[ts_.to_raw_iid(j)],
                              "Score": round(float(sc[j]), 4),
                          "Pred. rating":
                              round(m.predict(uid,
                              ts_.to_raw_iid(j)), 2) if
                              m.rating_model else "-",
                          "#ratings":
                              pop.get(ts_.to_raw_iid(j), 0),
                          "Liked in validation": "✓" if
                              ts_.to_raw_iid(j) in rel else
                              ""} for n, j in
                              enumerate(order, 1)])
```

อธิบายโค้ด

เหมือนระบบจริง: กรองเรื่องที่ดูแล้ว

- เรียง `scores(u)` จากมากไปน้อย ตัดเรื่องที่ดูแล้ว เอา Top-K
- แสดงชื่อ ปี แนว คะแนน ค่าทำนาย และจำนวนคะแนนของเรื่อง
- โหมด Validation: ติ๊ก ✓ ถ้าผู้ใช้ชอบเรื่องนั้นจริงในอนาคต

Part 11: GUI — รวมผลและตารางเทียบ

```
def gui_main(uid, name, k, mode, compare_models):
    _, ts_ = fitted_for(mode, name)
    md, hist = user_profile(uid, ts_); rec =
        recommend(uid, name, k, mode)
    if mode.startswith("Validation"):
        fut = te_main[te_main.user_id == uid]
        md += f"\n- Validation (อนาคต): {len(fut)}
คะแนน, ชอบ {int((fut.rating >=
            LIKE_T).sum())} เรื่อง → ✓ =แนะนำถูก"
    comp = pd.DataFrame({c: recommend(uid, c, k,
        mode).apply(lambda r: ("✓ " if r["Liked in
        validation"] else "") + r["Movie"],
        axis=1).values
                         for c in (compare_models or
                             [])}, index=[f"#{n}" for
                             n in range(1,
                             int(k)+1)]).T.reset_index
                             (names="Model")
    return md, hist, rec, comp
```

อธิบายโค้ด

หนึ่งปุ่ม สี่ผลลัพธ์

- ข้อมูลผู้ใช้ + ประวัติ + รายการแนะนำ
- โหมด Validation เพิ่มจำนวนคะแนน/เรื่องที่ชอบในอนาคต
- comp: Top-K ของอัลกอริทึมที่ติ๊กเลือก เทียบกันในตารางเดียว (✓ = แนะนำถูก)

Part 11: GUI — ทำนายคะแนนรายเรื่อง

```
def gui_predict(uid, movie, mode):
    iid = movie.split(" | ")[0]
    true = ratings[(ratings.user_id == uid) &
        (ratings.item_id == iid)].rating
    rows = [{"Model": n, "Predicted rating":
        round(fitted_for(mode, n)[0].predict(uid, iid),
        3)} for n in RATING_MODELS]
    return (f"จริง: {true.iloc[0] if len(true) else
        'ยังไม่เคยให้คะแนน'}", pd.DataFrame(rows))
```

อธิบายโค้ด

เทียบค่าทำนายทุกอัลกอริทึม

- รับภาพยนตร์จาก dropdown รูปแบบ `id | title`
- ค่าทำนายจาก 12 อัลกอริทึมที่ทำนายคะแนนได้
- แสดงค่าจริงถ้าผู้ใช้เคยให้คะแนน ⟶ เห็นว่าใครทำนายใกล้

Part 11: GUI — จัดหน้าด้วย gr.Blocks

```
with gr.Blocks(title="Chapter 5 – CF on MovieLens-100K (Surprise)") as demo:
    gr.Markdown("## Chapter 5 – Collaborative Filtering on MovieLens-100K
        (Surprise)")
    with gr.Row():
        dd_u = gr.Dropdown(USER_CHOICES, value=SAMPLE_USERS[0],
            label="User")
        dd_m = gr.Dropdown(MODEL_NAMES, value="SVD", label="Algorithm")
        sl_k = gr.Slider(5, 20, 10, step=1, label="Top-K")
        rd = gr.Radio(["Validation (S4 temporal)", "Full data"],
            value="Validation (S4 temporal)", label="Mode")
    with gr.Tab("Recommend"):
        cg = gr.CheckboxGroup(MODEL_NAMES, value=["Popularity",
            "ItemKNN-Baseline", "SVD", "EASE"], label="เทียบกับอัลกอริทึม")
        bt = gr.Button("Recommend", variant="primary")
        md = gr.Markdown()
        with gr.Row():
            h = gr.Dataframe(label="User history (top-rated)")
            r = gr.Dataframe(label="Recommended movies")
        c = gr.Dataframe(label="Top-K comparison")
        bt.click(gui_main, [dd_u, dd_m, sl_k, rd, cg], [md, h, r, c])
    with gr.Tab("Predict rating"):
        dd_mv = gr.Dropdown(MOVIE_CHOICES, value=MOVIE_CHOICES[0],
            label="Movie")
        bp = gr.Button("Predict")
        out_t = gr.Markdown(); out_df = gr.Dataframe(label="Predicted
            rating by algorithm")
        bp.click(gui_predict, [dd_u, dd_mv, rd], [out_t, out_df])

demo.launch(debug=False)
```

อธิบายโค้ด

แอปสองแท็บ

- แถวบน: User, Algorithm, Top-K, Mode ใช้ร่วมกันทุกแท็บ
- แท็บ **Recommend**: เลือกอัลกอริทึมที่ต้องการเทียบด้วย CheckboxGroup
- แท็บ **Predict rating**: เลือกภาพยนตร์แล้วกด Predict
- แนวทางใช้: ผู้ใช้ 56 → EASE vs SVD vs ItemKNN ดูว่าใครแนะนำหนังแอ็กชันที่เขาชอบ

สรุปแนวคิดของ Source Code LAB 2

Take-away

- Surprise ให้อัลกอริทึมครบ แต่ **ต้องห่อ** ให้มี `scores()` จึงประเมินแบบ ranking และ beyond-accuracy ได้
- วิธีเลือกกลุ่มตัวอย่างเปลี่ยนตัวเลขมาก: Temporal ยากกว่า Random, Global Temporal มี cold start, Sampled metric สูงเกินจริง
- บน MovieLens โมเดลที่เรียนพารามิเตอร์ชนะด้าน RMSE (SVD++, KNNBaseline) แต่ **EASE / Popularity ชนะด้าน ranking**
- Beyond-Accuracy เผยว่า kNN แนะนำหนังหายาก ส่วน Popularity แนะนำซ้ำ ๆ — ต้องอ่านคู่กับความแม่น
- Paired bootstrap + ตัวอย่าง Output รายผู้ใช้ ทำให้ข้อสรุปมีหลักฐานและเห็นภาพ

LAB 3 ต่อยอดเป็น Pipeline: เตรียมข้อมูล → ปรับ Hyperparameter (RMSE vs nDCG) → ทดสอบครั้งเดียว → ใช้งานจริง